

Árvores Binárias de Busca

A estrutura que equilibra busca, inserção e remoção.

1	2	3	4	5	6
Motivação	Conceito	Operações	Percursos	Código C++	STL

Lista encadeada

A lista encadeada é boa para inserção e remoção – basta ajustar ponteiros. Mas buscar um elemento exige percorrer do início ao fim.

**Inserção / Remoção** **$O(1)$**

Ajusta ponteiros – rápido independente do tamanho.

Busca **$O(n)$**

Percorre do início ao fim – lento para listas grandes.

Ordenação **$O(n^2)$**

Não mantém ordem – ordenar exige algoritmo extra.

Busca binária

A busca binária é rápida – divide o espaço de busca pela metade a cada passo. Mas exige que o array esteja sempre ordenado.

1	3	5	8	12	15	20	25
---	---	---	---	----	----	----	----

Buscando 12: $\text{meio}=[8,12] \rightarrow \text{encontrado!}$

Busca

$O(\log n)$

Divide pela metade a cada passo – muito eficiente.

Exige array ordenado

$O(n \log n)$

Inserir um elemento exige reordenar o array inteiro.

Inserção / Remoção

$O(n)$

Mover elementos para manter a ordem é caro.

Tabela hash

A tabela hash é excelente para busca. Mas não mantém ordem e pode ser custosa quando há muitas inserções e remoções que causam rehashing.

0	→ 5
1	→ 12
2	
3	→ 3,8
4	
5	→ 20
6	→ 1
7	

Busca $O(1)^*$ – Acesso direto pela chave – muito rápido.

Inserção / Remoção $O(1)^*$ – Em média rápido – mas colisões e rehashing custam.

Ordem / Percurso $O(n \log n)$ – Não mantém ordem.
Percorrer em ordem exige ordenar depois.

Pior caso $O(n)$ – Muitas colisões degradam a performance.

Onde a árvore entra

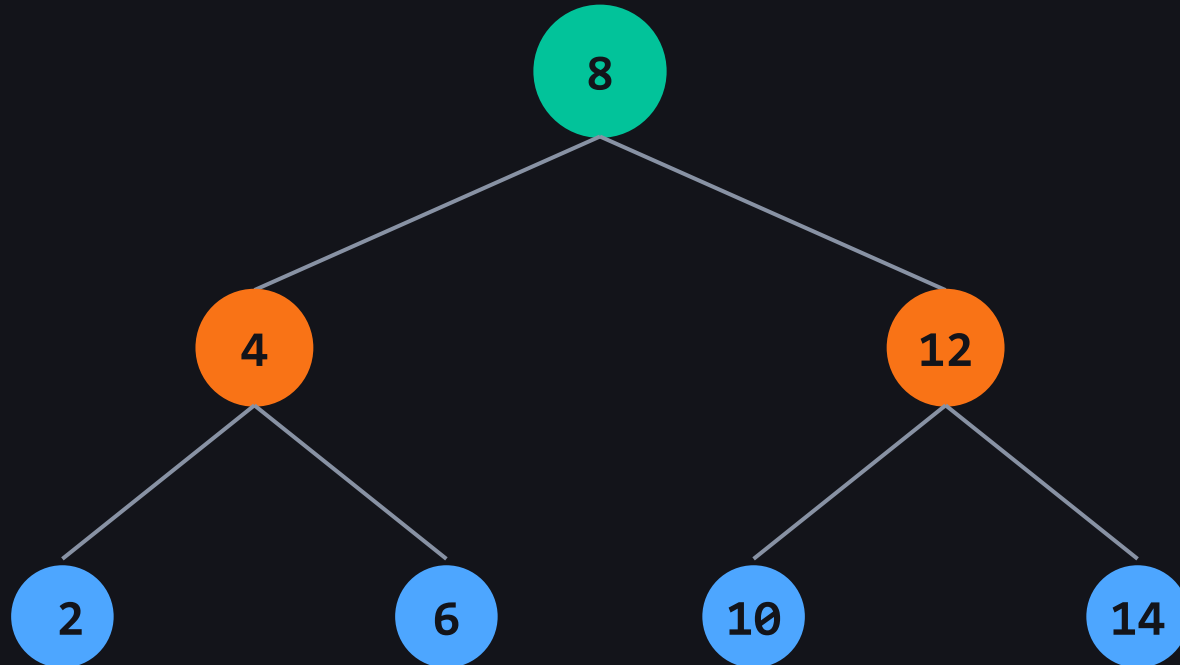
Nenhuma estrutura que vimos resolve os três ao mesmo tempo com eficiência.

	Busca	Inserção	Remoção	Ordenado?
Lista encadeada	$O(n)$	$O(1)$	$O(1)$	Não
Busca binária	$O(\log n)$	$O(n)$	$O(n)$	Sim
Tabela hash	$O(1)^*$	$O(1)^*$	$O(1)^*$	Não
Árvore bin. de busca	$O(\log n)$	$O(\log n)$	$O(\log n)$	Sim

A árvore binária de busca equilibra os três – e mantém os dados em ordem.

O que é uma árvore?

Uma árvore é uma estrutura de dados hierárquica formada por nós conectados por arestas. Cada nó pode ter filhos, mas cada filho tem apenas um pai.



Raiz

O único nó sem pai. É o ponto de entrada da árvore – toda travessia começa por ela. Toda árvore tem exatamente uma raiz.

Nó

Cada elemento da árvore. Contém um valor e referências para seus filhos. Um nó pode ter zero, um ou dois filhos em uma árvore binária.

Folha

Nó sem filhos – seus ponteiros esquerdo e direito apontam para null. Sempre está no nível mais externo da árvore.

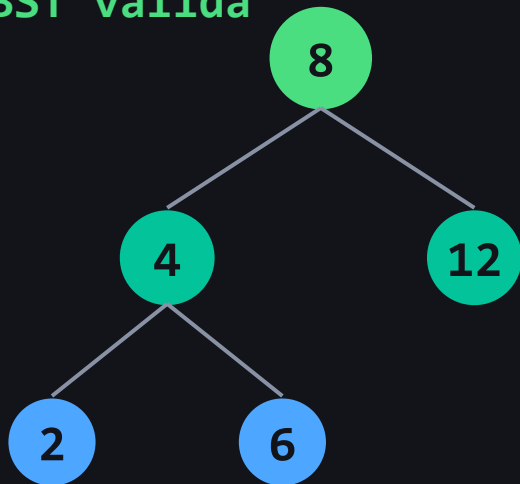
BST – a propriedade fundamental

Para todo nó N:

subárvore ESQUERDA → apenas valores MENORES que N

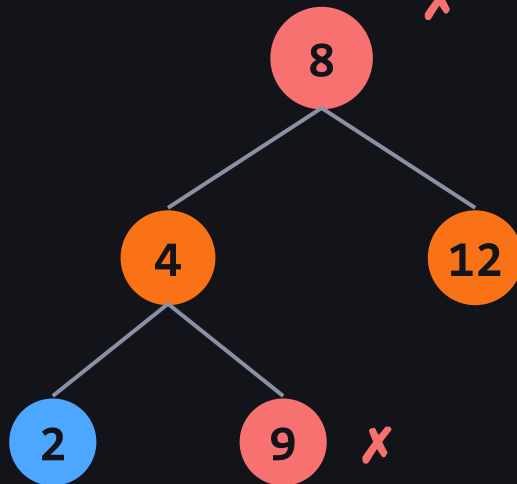
DIREITA → apenas valores MAIORES que N.

✓ BST válida



$2 < 4 < 6$ – todos menores que 8 ✓

✗ BST inválida



$9 > 8$ mas está na subárvore esquerda – viola a propriedade!

Uma árvore que não respeita essa propriedade pode ter a estrutura de uma árvore binária, mas não é uma BST – a busca eficiente não funciona.

Como encontrar um valor

A propriedade da BST torna a busca simples: a cada nó, decidimos se vamos para a esquerda ou direita – nunca precisamos olhar os dois lados.

Buscando o valor 10:

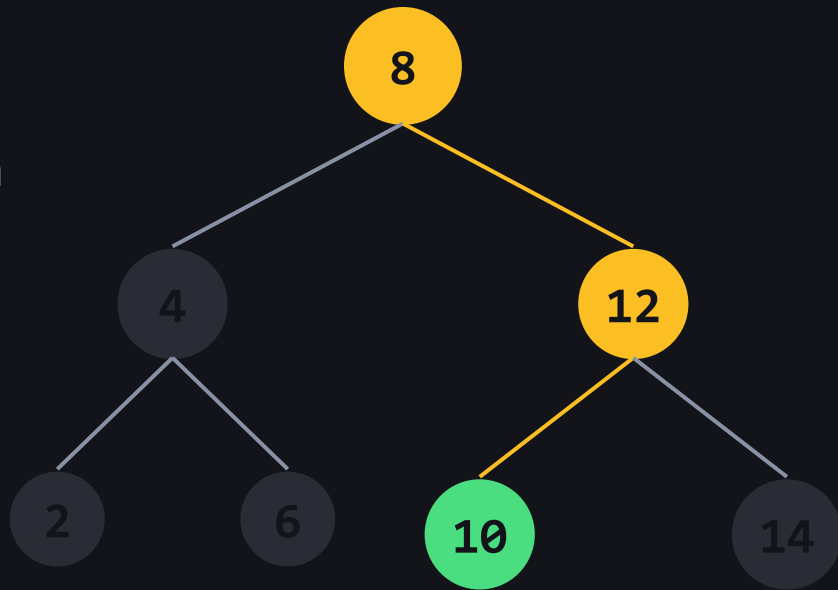
1 Nó 8: $10 > 8 \rightarrow$ vai para a direita

2 Nó 12: $10 < 12 \rightarrow$ vai para a esquerda

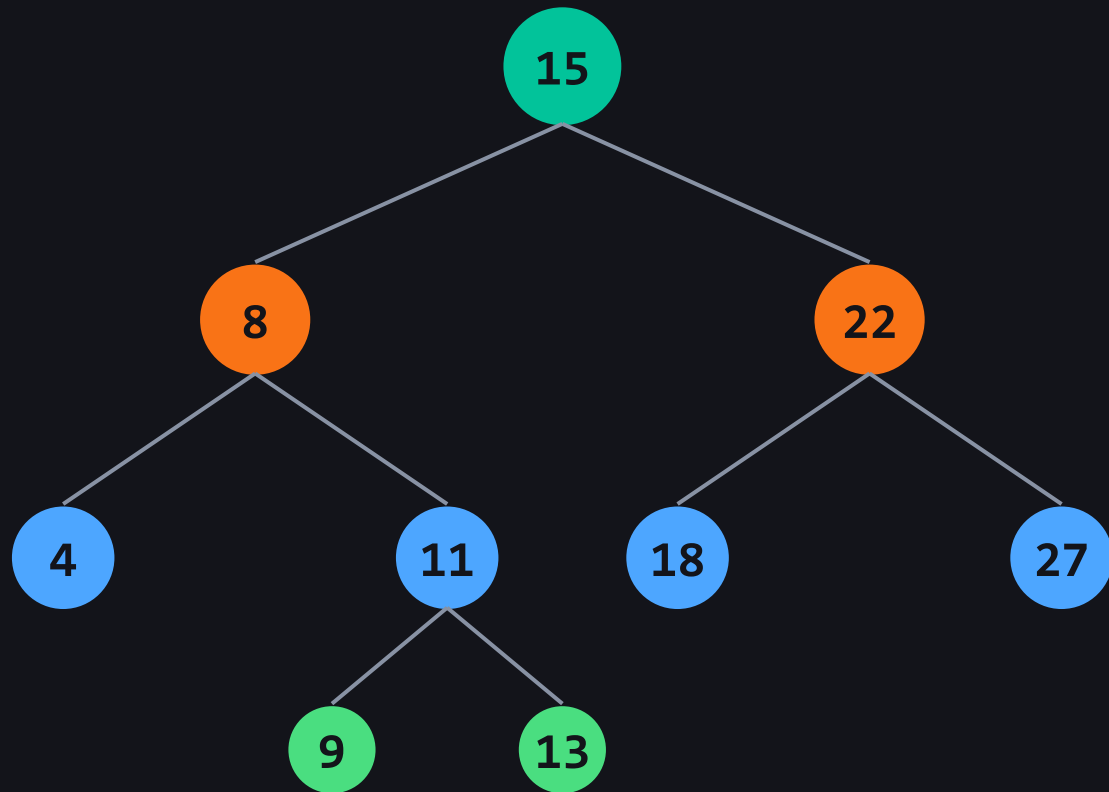
3 Nó 10: $10 == 10 \rightarrow$ encontrado!

Algoritmo:

se valor == nó $\rightarrow$ encontrado | se valor < nó $\rightarrow$ busca na esquerda |
se valor > nó $\rightarrow$ busca na direita



Para cada valor abaixo, percorra a árvore e anote os nós visitados até encontrar o valor ou concluir que ele não existe.



Busque:

13

22

9

20

4

Para cada busca: quais nós foram visitados? Quantas comparações foram necessárias?

Inserção – como adicionar um valor

Inserir é como buscar: seguimos o caminho até encontrar um espaço vazio (null) onde o novo nó se encaixa mantendo a propriedade da BST.

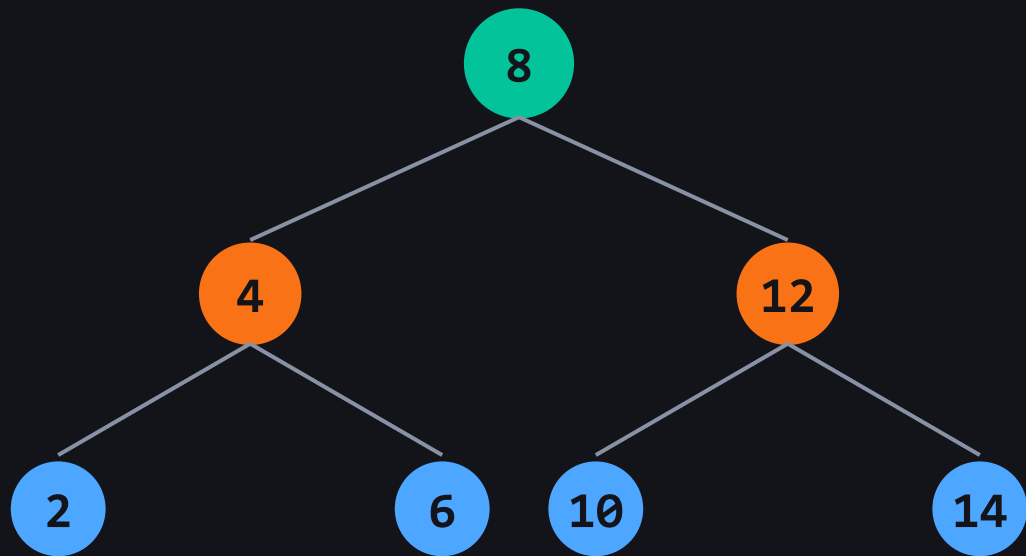
Inserindo o valor 7:

- 1 7 > 8? Não → vai para a esquerda (nó 4).
- 2 7 > 4? Sim → vai para a direita (nó 6).
- 3 7 > 6? Sim → vai para a direita. Posição é null → insere aqui!

Resultado após inserir 7:

Em-ordem – esquerda, raiz, direita

Visita os nós na ordem: subárvore esquerda → nó atual → subárvore direita. Resultado: valores em ordem crescente.



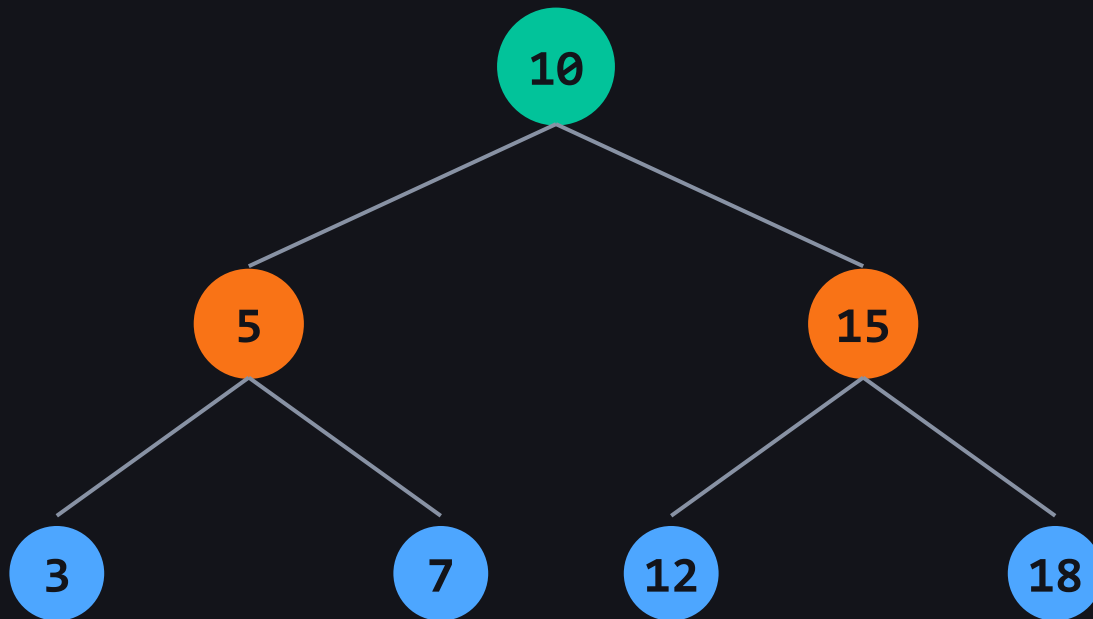
Ordem de visita:



Em-ordem em uma BST sempre produz os valores em ordem crescente – a árvore mantém os dados ordenados implicitamente.

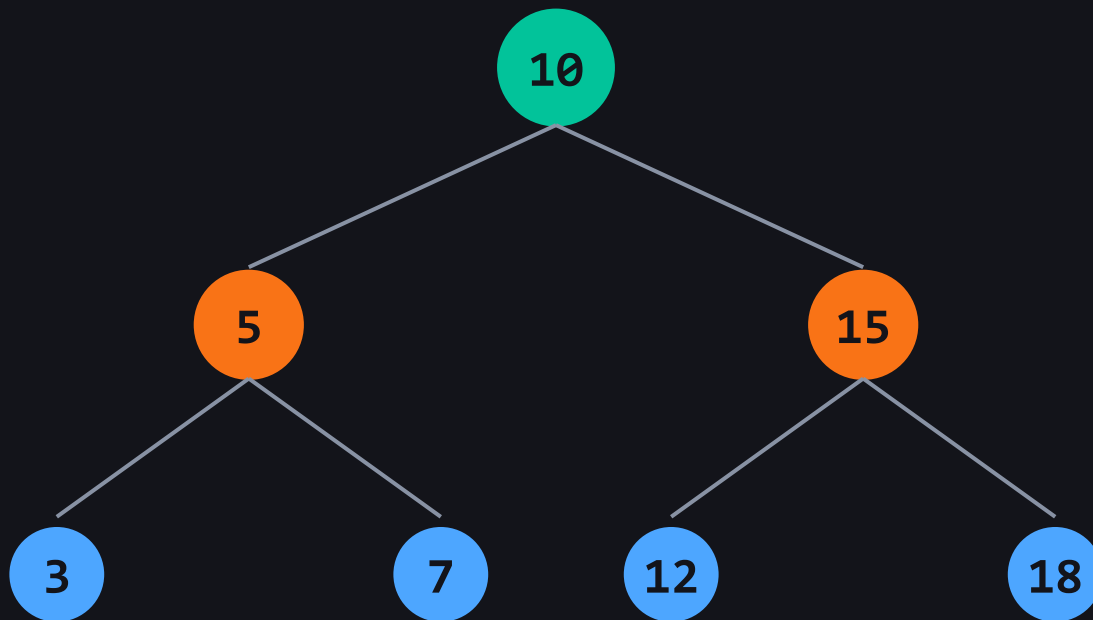
Exercício – Em-ordem

Dado a árvore abaixo, escreva a sequência de visita do percurso em-ordem.



Em-ordem: _____

Resposta – Em-ordem

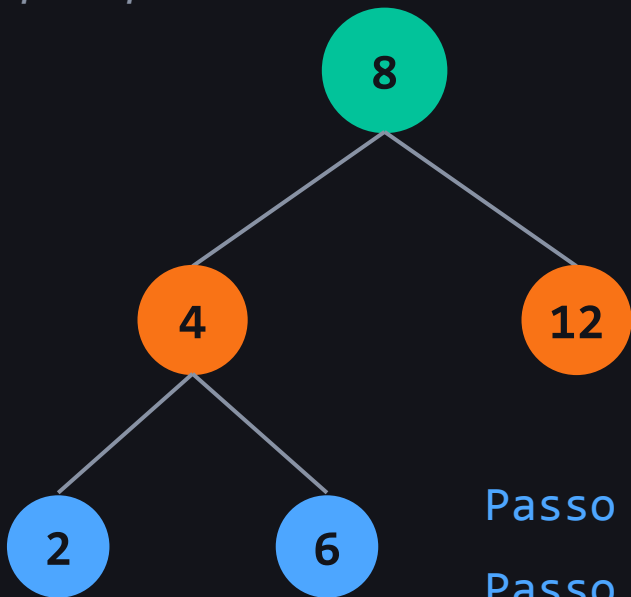


Em-ordem: 3, 5, 7, 10, 12, 15, 18

Esquerda → raiz → direita. Sempre produz valores em ordem crescente.

Pré-ordem – raiz → esquerda → direita

Visita o nó atual antes de descer para os filhos. O nó pai é sempre processado antes de qualquer filho.



Resultado:



Passo 1: visita 8 (raiz)

Passo 2: desce esquerda → visita 4

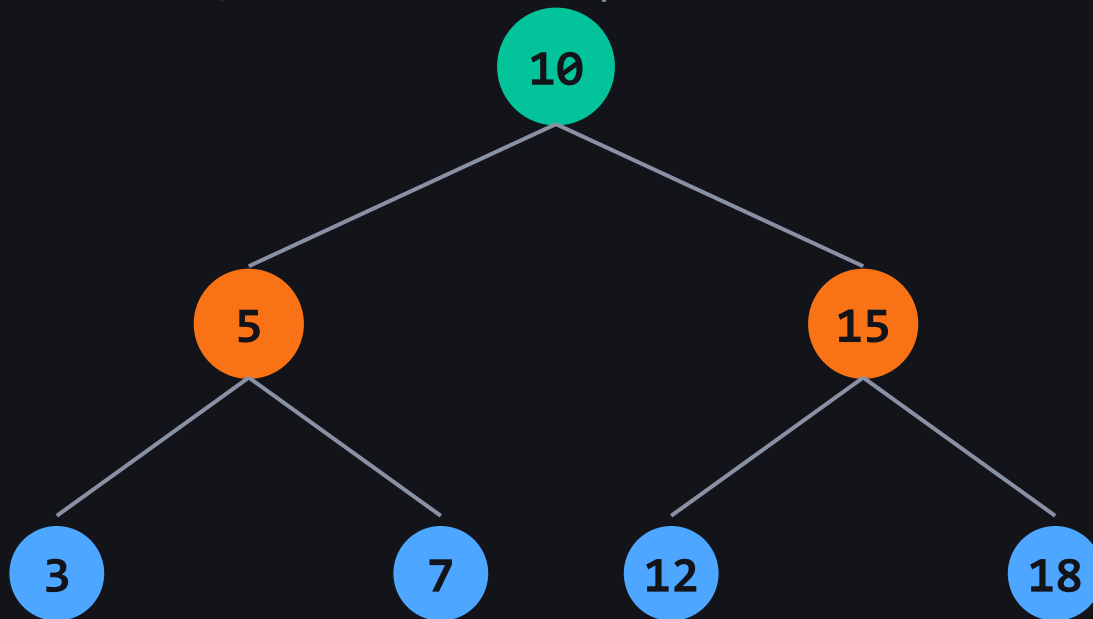
Passo 3: desce esquerda → visita 2 (folha)

Passo 4: volta, direita de 4 → visita 6 (folha)

Passo 5: volta à raiz, direita → visita 12

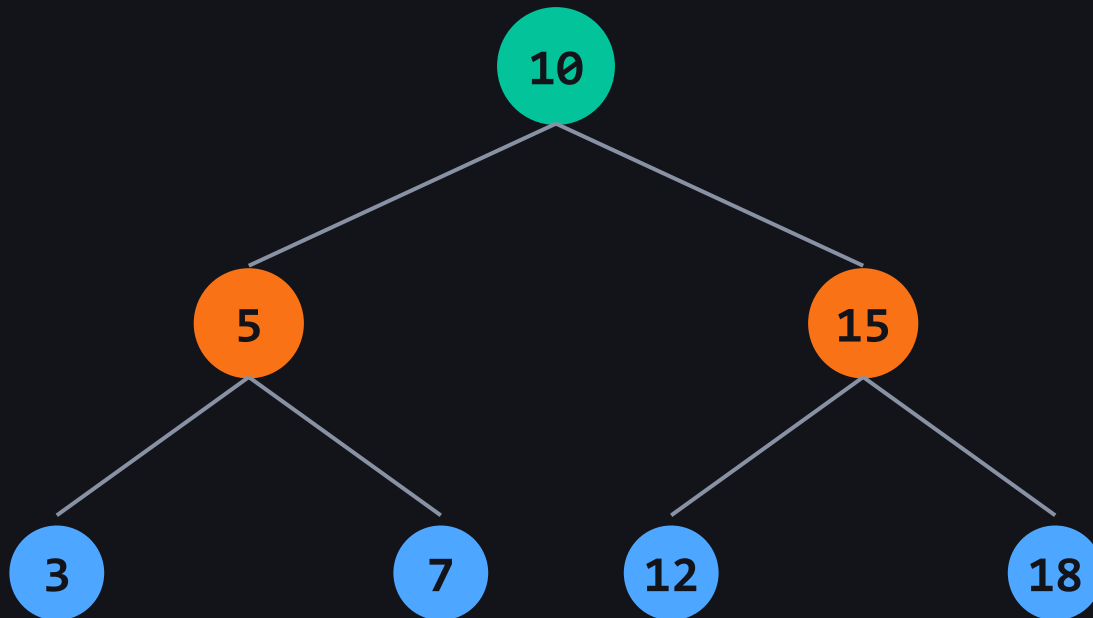
Exercício – Pré-ordem

Dado a árvore abaixo, escreva a sequência de visita do percurso pré-ordem.



Pré-ordem: _____

Resposta – Pré-ordem

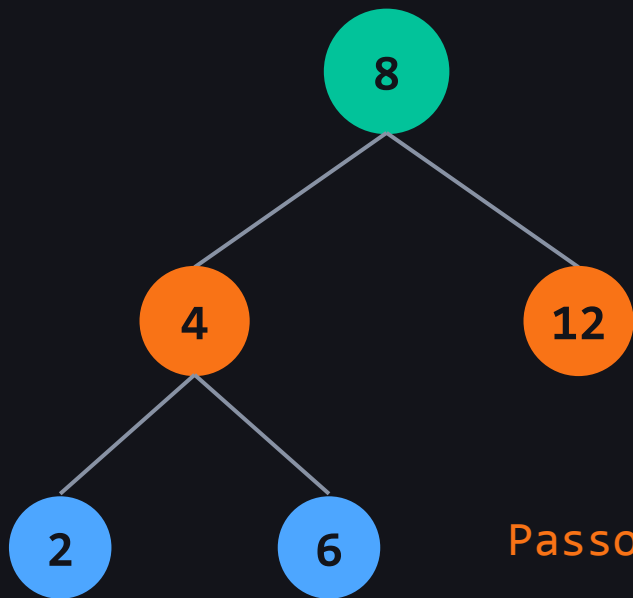


Pré-ordem: 10, 5, 3, 7, 15, 12, 18

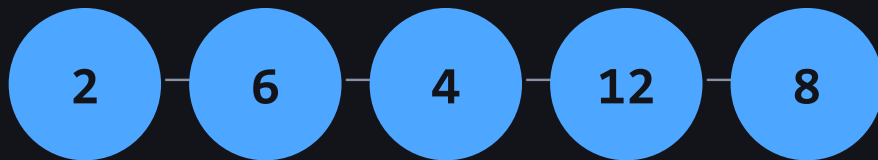
Raiz → esquerda → direita. A raiz é sempre o primeiro elemento visitado.

Pós-ordem – esquerda → direita → raiz

Visita o nó atual depois de processar todos os filhos. Os nós filhos são sempre processados antes do pai.



Resultado:



Passo 1: desce até a folha mais à esquerda → visita 2

Passo 2: direita de 4 → visita 6 (folha)

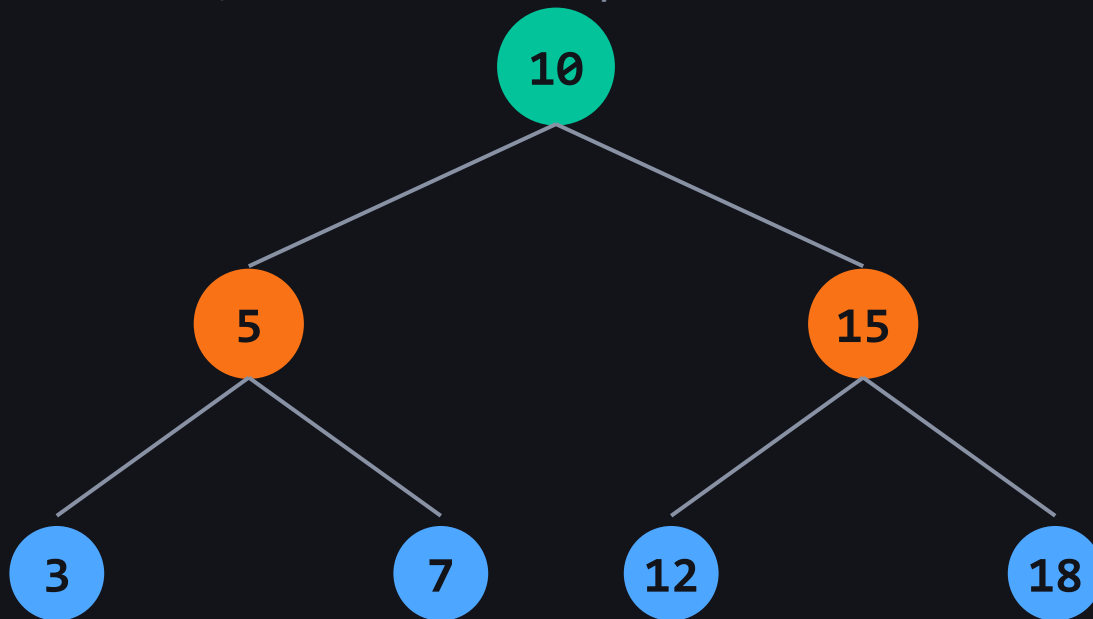
Passo 3: filhos de 4 prontos → visita 4

Passo 4: direita da raiz → visita 12 (folha)

Passo 5: todos os filhos prontos → visita 8 (raiz)

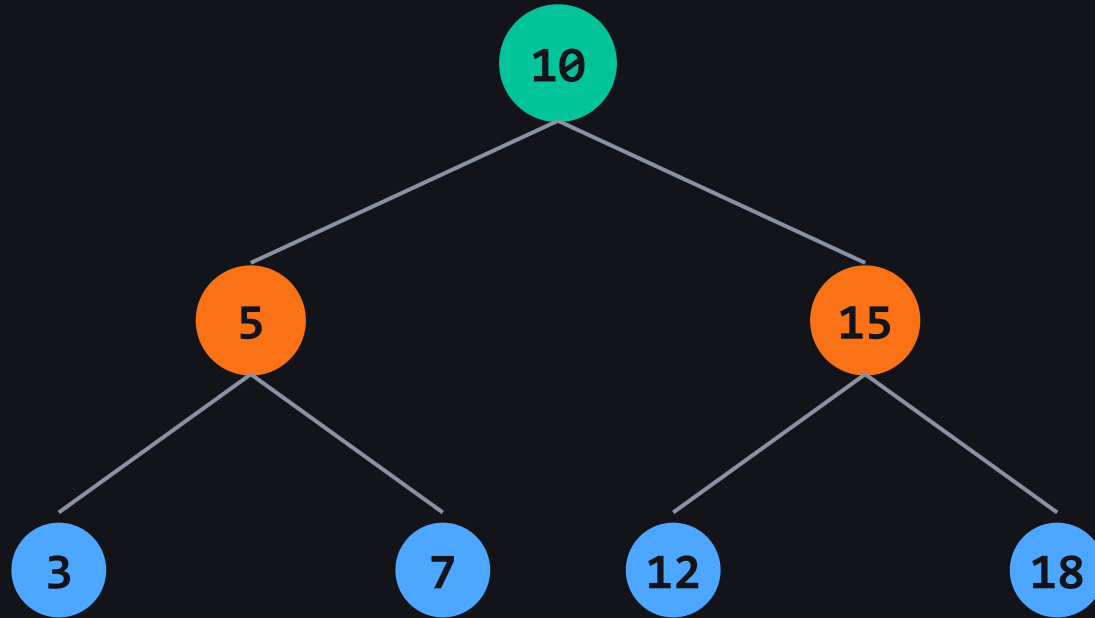
Exercício – Pós-ordem

Dado a árvore abaixo, escreva a sequência de visita do percurso pós-ordem.



Pós-ordem: _____

Resposta – Pós-ordem



Pós-ordem: 3, 7, 5, 12, 18, 15, 10

Esquerda → direita → raiz. As folhas aparecem primeiro, a raiz por último.

Altura

Número de arestas no caminho mais longo da raiz até uma folha. Uma árvore com apenas a raiz tem altura 0.

Profundidade

Número de arestas de um nó até a raiz. A raiz tem profundidade 0. Um filho da raiz tem profundidade 1.

Subárvore

Qualquer nó e todos os seus descendentes formam uma subárvore. A BST aplica sua propriedade recursivamente a cada subárvore.

Exercício – construindo e modificando a árvore

Partindo de uma árvore vazia, execute as operações abaixo em ordem e desenhe o estado da árvore após cada uma.

1. Inserir 15

2. Inserir 8

3. Inserir 20

4. Inserir 5

5. Inserir 12

6. Inserir 17

7. Buscar 12

8. Remover 8

9. Inserir 10

10. Buscar 10

Após inserir os 6 primeiros valores: qual nó é a raiz? Qual é a altura da árvore? Quais são as folhas?

Remoção – os três casos

Caso 1 – Nó folha (sem filhos)

Remove o nó. O pai passa a apontar para null.

Caso 2 – Nó com um filho

Remove o nó e conecta o filho diretamente ao pai.

Caso 3 – Nó com dois filhos

Substitui pelo sucessor em-ordem (menor da subárvore direita) e remove o sucessor.

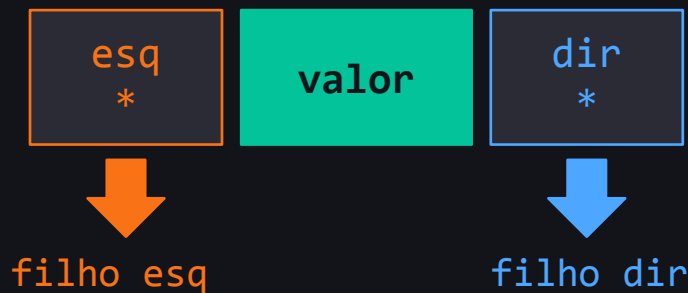
Implementação – estrutura do nó

```
#include<iostream>
using namespace std;

struct No {
    int valor;
    No* esq;  // filho esquerdo
    No* dir;  // filho direito

    // construtor
    No(int v) {
        valor = v;
        esq = nullptr;
        dir = nullptr;
    }
};
```

Representação visual do nó:



A árvore inteira é acessada apenas pelo ponteiro para a raiz. Se a raiz for nullptr, a árvore está vazia.

Implementação – inserção

```
// Insere recursivamente mantendo a propriedade BST
No* inserir(No* raiz, int valor) {
    // Caso base: posição vazia – cria o nó aqui
    if (raiz == nullptr)
        return new No(valor);

    // Valor menor → desce pela esquerda
    if (valor < raiz->valor)
        raiz->esq = inserir(raiz->esq, valor);

    // Valor maior → desce pela direita
    else if (valor > raiz->valor)
        raiz->dir = inserir(raiz->dir, valor);

    // Valor igual → não insere duplicata
    return raiz; // retorna o nó atual atualizado
}
```

Implementação – pré-ordem e em-ordem

```
// Pré-ordem
// raiz → esquerda → direita
void preOrdem(No* raiz) {
    if (raiz == nullptr) return;
    cout << raiz->valor << ' ';
    preOrdem(raiz->esq);
    preOrdem(raiz->dir);
}
// Saída para a árvore exemplo:
// 8 4 2 6 12 10 14
```

```
// Em-ordem
// esquerda → raiz → direita
void emOrdem(No* raiz) {
    if (raiz == nullptr) return;
    emOrdem(raiz->esq);
    cout << raiz->valor << ' ';
    emOrdem(raiz->dir);
}
// Saída para a árvore exemplo:
// 2 4 6 8 10 12 14 ← em ordem!
```

A ordem das três linhas dentro da função define tudo – a única diferença entre os percursos é quando a raiz é visitada em relação aos filhos.

Implementação – pós-ordem

```
// Pós-ordem: esquerda → direita → raiz
void posOrdem(No* raiz) {
    if (raiz == nullptr) return;
    posOrdem(raiz->esq);
    posOrdem(raiz->dir);
    cout << raiz->valor << ' ';
}

// Saída: 2 6 4 10 14 12 8
```

Por que pós-ordem?

Deletar a árvore

Libera memória dos filhos antes do pai – seguro para desalocar.

Avaliar expressões

Em árvores de expressão matemática, operandos antes do operador.

Calcular tamanhos

Calcula o tamanho de subárvores antes de reportar ao pai.

Sistema de cadastro de produtos

Uma loja precisa de um sistema para gerenciar seus produtos por código. Os códigos são números inteiros únicos e o sistema deve manter sempre a ordem para facilitar a busca.

1	Cadastrar produto:	Insere um código na BST.
2	Buscar produto:	Retorna se o código existe. Exibe o caminho percorrido.
3	Listar em ordem:	Exibe todos os códigos em ordem crescente
4	Remover produto:	Remove um código mantendo a propriedade da BST.
5	Quantidade de produtos	Calcular e exibir o total de produtos cadastrados

Teste com os códigos: 50, 30, 70, 20, 40, 60, 80, 10, 35 –
nessa ordem.